SaltShield — Hackachusetts Submission

Track: ClimateTech

Live site: https://evelynkatayose.github.io/saltshield/

Repeatable demo: https://evelynkatayose.github.io/saltshield/?demo=1

Source: https://github.com/EvelynKatayose/saltshield

Demo video: https://youtu.be/e0FRNgocqTg

No login is required. The site is public and free to use for judging and testing.

1. Project description (features & functionality)

The problem

Three Massachusetts towns — Holden, Sterling, and West Boylston — share part of the Wachusett Reservoir watershed, a public drinking-water source. Each town runs its own winter road-salt operation and its own equipment budget. Road salt keeps roads safe, but chloride is a long-term concern for the shared water supply. Town lines stop at the border; the watershed does not. The practical question nobody owns is: **where can a limited winter-equipment budget do the most good for the water we all drink?**

What SaltShield does

SaltShield is an offline-first environmental planning tool that connects **official public GIS road inventory** to a transparent budget-allocation model, so planners from three towns can compare independent spending against a shared ("pooled") budget and see the tradeoff honestly.

Immersive experience landing page

- A full-width winter story (hero, scroll-guided Snow → Salt → Water Canvas 2D illustration) that explains *why* the problem matters before showing numbers.
- A **live budget comparison**: one budget slider drives two synchronized maps (Independent vs. Shared), showing the exact planning cells each policy funds.
- Cursor-responsive snow, salt scattering, and water ripples — all explicitly illustrative and firewalled from the quantitative model.

Four-section planning workspace

- **Overview** — watershed map, three-town road inventory, and a live decision brief.
- **Investments** — edit budget, salt baseline, project costs, coverage and enablement; compare five funding rules (independent, pooled, pooled-with-every- town, equal-actual-spend, and the participation-constrained equal-spend case); choose Zone A or all A/B/C roads as the objective.
- **Road Coverage** — link equipment purchases to whole 1 km road cells under a simulated service-reach limit, with an exact knapsack solver and nine reach sensitivity cases.
- **Evidence** — 13 historical DCR grant awards (FY2021–FY2026, $188,251), five winters of ERA5 regional weather context, and explicit measurement rules.

Honesty by design

- Geometry and road lengths are **measured** (EPSG:26986) and conserved; prices, reduction rates, baselines, coverage and capacities are **explicitly simulated and labeled**. The tool never claims verified water-quality improvements.

- Every output carries source SHA-256 hashes and parameter fingerprints; stale or tampered inputs are rejected rather than silently used.
- Exact budget optimization enumerates all 27 none/CAL/BRINE combinations; the spatial model uses certified MILP (SciPy/HiGHS) with a zero-gap requirement.

Engineering highlights

- The entire shipped report is a **single self-contained HTML file** — no CDN, no framework, no remote fonts, no runtime network calls. It runs fully offline.
- Accessibility: keyboard navigation, focus highlighting, reduced-motion support, a no-Canvas static fallback, and narrow-screen layouts.
- Reproducible build pipeline (Python analysis → self-contained report → staged GitHub Pages site) with Python unit tests and Node parity/UI tests that execute the exact scripts shipped in the report.

Data sources (unique public resources)

- MassGIS / MassDOT Roads, Municipalities, Surface-Water Supply Watersheds, and Protection Areas (Zones A/B/C).
- ERA5 reanalysis via Open-Meteo (CC BY 4.0) — five complete winters.
- MA DCR historical grant records for the three study towns.

Who benefits and how it scales

Municipal public-works planners and watershed stakeholders get a shared, neutral basis for a conversation that usually never happens across town lines. The pipeline is data-driven and parameterized: the same approach generalizes to any multi-jurisdiction shared watershed by swapping the GIS extract and parameter tables.

2. How Kiro was used

Tool used: Kiro IDE. This was our first time building with Kiro, and the entire project was developed inside the Kiro IDE. We did not use the CLI, Web, or Crew surfaces for this project, so we cannot honestly compare them here.

Spec-driven development

We used Kiro's **spec workflow** (requirements → design → tasks) to drive a focused, reviewable change rather than open-ended "vibe coding." The spec for the experience-page visual enhancement lives at `.kiro/specs/experience-visual-enhancement/`:

- `requirements.md` — written in EARS-style acceptance criteria, including a hard requirement that purely-visual changes must **not** alter the quantitative model, accessibility behavior, or the public base path.
- `design.md` — captured the build architecture (how CSS/JS get concatenated into the self-contained report) so edits were grounded in how the code actually ships.
- `tasks.md` — a checklist with explicit requirement traceability and a task dependency graph.

Why the spec helped vs. vibe coding: writing the acceptance criteria first forced the "do no harm" constraints (model isolation, reduced-motion, `/saltshield/` base path) to be explicit *before* any code changed. That turned an ambiguous "make it prettier" request into a bounded change with a clear done-definition, and every edit mapped back to a requirement. Vibe coding would likely have drifted into the model layer or broken the embedded-script tests.

Steering docs

We added three **always-included steering files** in `.kiro/steering/`:

- `product.md` — product purpose, audience, public URLs, and the honesty principles.
- `tech.md` — the exact build/test pipeline and the gotcha that `build_report.py` refuses to run with stale phase outputs.
- `structure.md` — directory conventions and the single most important project rule: **never blur measured vs. simulated**, and never hand-edit generated files.

Strategy that made the biggest difference: encoding the "measured vs. simulated" boundary and the "self-contained output / edit the source, not the build artifact" rule into steering. Those two rules are the project's core discipline, and having them always in context kept every subsequent change aligned without re-explaining them each time.

Agent hooks

We automated three workflows in `.kiro/hooks/`:

- `rebuild-report-on-save` (PostFileSave) — when a presentation-layer source file (`experience.*`, `experience_live.css`, `winter_scene.js`, `workspace.*`, `report_template.html`) is saved, it reminds the agent to rebuild the self-contained report and re-stage the site before running tests. This directly prevents the most common mistake in this repo: running Node tests against a stale built report.
- `test-experience-after-build` (PostFileSave on the built report) — runs `node test_experience.cjs` automatically after a rebuild.
- `guard-deployment-config` (PreToolUse on write tools) — intercepts any attempt to edit `deployment/site.json` and asks for confirmation, protecting the public GitHub Pages domain/base path from accidental change.

How hooks improved the process: the build→stage→test ordering in this project is strict and easy to get wrong. The hooks encode that ordering as automation instead of tribal knowledge, and the PreToolUse guard turns a high-risk edit (changing the live domain) into a deliberate, confirmed action.

What we did not use (stated honestly)

- **MCP:** not used. The project is fully offline by design (no runtime network calls, no external services), so there was no external capability we needed to bridge in via MCP for this build.
- **Property-based testing:** we did not implement formal PBT. Our testing is cross-language *parity* testing — `test_planning.cjs` and `test_joint.cjs` compare the browser JavaScript solver against the Python implementation across 152 scenarios / 1,064 policy results and verify deterministic, bounded particle state — plus Python `unittest` suites. A natural next step would be PBT over the measurement invariants (length conservation, disjoint zone accounting, budget feasibility).
- **Kiro Powers:** none installed for this project.

We've chosen to report our Kiro usage accurately rather than overstate it. The depth is concentrated in spec-driven development, steering, and hooks working together to enforce this project's unusual "measured vs. simulated" and "self-contained build" disciplines.

3. Testing / run instructions

The deployed site needs no setup — open the live URL. To build and verify locally:

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Analysis pipeline (offline, uses cached data)
python run_analysis.py
python build_evidence.py
python phase2_model.py
python prepare_parameters.py
python planning_v1.py
python build_weather.py
python spatial_priority.py
python joint_planning.py
python build_report.py      # assembles results/SaltShield_Wachusett.html
python build_site.py        # stages the public site into docs/

# Verification
python -m unittest test_phase2 test_phase3 test_planning_v1 test_joint_planning -v
node test_ui.cjs
node test_planning.cjs      # Python/JavaScript numerical parity (needs python3)
node test_joint.cjs
node test_joint_brief.cjs
node test_experience.cjs
node test_demo.cjs
node test_routing.cjs
```

Then open `results/SaltShield_Wachusett.html` directly in a browser — everything is embedded; no server or network connection is required.

4. Checklist against submission requirements

- [x] `.kiro/` at repo root, not in `.gitignore` (steering + specs + hooks)
- [x] Public GitHub repository with full source, assets, and instructions
- [x] Public, free, working hosted project for judging/testing
- [x] All materials in English
- [x] Challenge track identified (ClimateTech & Clean Energy)
- [x] Kiro write-up (section 2)
- [x] Team members added on the submission form (≤4)
- [x] Demo video (<3 min) uploaded publicly and linked above
- [x] Submission form fields completed on the Hackathon website